



Desafío Final: Implementación y ejecución del juego de una invasión Alién en el espacio a partir de C++

Estudiante: **Arlington Zahir Llerena**

CC: 72 277 711

Email: zahir.llerena@udea.edu.co

Número telefónico: + 57 300 726 22 82

Profesores: **AUGUSTO ENRIQUE SALAZAR JIMENEZ &**

KEVIN DAVID MARTINEZ ZAPATA

Grupo 003. Código: 2570201 – Informática II -

Facultad de Ingeniería: Telecomunicaciones Región – Carmen del Viboral –

Universidad de Antioquia

6 de Junio de 2026

Introducción

El proyecto Alien Invasion es un motor de juego bidimensional (vista cenital) implementado en C++ utilizando el framework Qt6 (Graphics View Framework). El sistema aplica los principios de Programación Orientada a Objetos (POO) para gestionar un entorno dinámico en tiempo real, donde la cinemática de los proyectiles, la lógica de enjambre de los alienígenas y la detección de colisiones son gestionadas por un orquestador central.

Objetivo técnico general

- Diseñar y construir un motor de juego de arquitectura modular basado en el paradigma de Programación Orientada a Objetos (POO) en C++ y Qt6 Graphics View Framework. El sistema implementa una simulación bidimensional de vista cenital que integra un gestor de entidades dinámicas (naves, invasores, proyectiles), un sistema de control de eventos en tiempo real y una máquina de estados para la gestión de flujo de juego. El motor garantiza la eficiencia computacional en la resolución de colisiones mediante lógica booleana, el procesamiento determinista de estados cinemáticos y la escalabilidad del sistema mediante

una estructura de clases altamente cohesiva, cumpliendo con los estándares de rendimiento y mantenibilidad requeridos para el curso de Informática II.

Objetivos específicos

Para alcanzar el objetivo anterior, pretendemos dividir el proyecto en diversas fases que en nuestra opinión permiten una estructura adecuada de la iniciativa, más aún cuando debemos ser flexibles en la propuesta de código puesto que pretendemos apelar a clases y funciones modulares que se complementen entre sí. Este diseño modular, brinda muchas ventajas en términos de edición puesto que, si durante el proceso de construcción y ejecución de estos módulos se detecta una tarea inadecuada por parte de estos fragmentos de código, se puede editar el módulo correspondiente, sin afectar la integridad de la totalidad del algoritmo. Ante lo anterior y bajo la propuesta de diseño inicial podemos mencionar algunos objetivos preliminares que dentro de nuestra propuesta consideramos sumamente importantes:

- Arquitectura de Control y Orquestación: Establecer un ciclo de ejecución principal (Main Loop) robusto mediante la integración de un temporizador de alta precisión (QTimer), coordinado por una clase de configuración centralizada (Settings). Esta fase define la estructura de delegación donde el punto de entrada `main()` orquesta clases especializadas responsables del despliegue del escenario, la lógica de control del jugador y el comportamiento de la horda enemiga.
- Gestión Dinámica de Recursos y Ciclo de Vida: Implementar contenedores eficientes para la administración de proyectiles, utilizando las capacidades de gestión de memoria de C++ para instanciar y destruir objetos de forma dinámica. Se integrarán rutinas de recolección de instancias fuera de dominio (off-screen cleanup) para garantizar la optimización de los recursos del sistema durante la ejecución en tiempo real.
- Algoritmia de Generación y Disposición Espacial: Diseñar e implementar un motor de despliegue de flotas basado en una lógica de posicionamiento computado. Se utilizarán estructuras de datos dinámicas para calcular la matriz de entidades invasoras, optimizando el uso del espacio bidimensional y respetando las restricciones geométricas del Canvas mediante una gestión precisa de coordenadas y márgenes.
- Cinemática Colectiva y Lógica de Frontera: Desarrollar el sistema de movimiento de la horda mediante vectores de dirección y estados de traslación en el eje de ordenadas (Y). Esta fase integra algoritmos de detección de colisiones con los límites del plano (condiciones de frontera), empleando punteros y paso de referencia para asegurar una manipulación eficiente de la información de estado de cada entidad invasora.

FUNDAMENTOS DE LA ARQUITECTURA DEL SISTEMA

El motor *Alien Invasion* ha sido diseñado bajo un paradigma de programación modular orientada a objetos, donde la cohesión entre componentes garantiza una simulación robusta y escalable. La arquitectura se fundamenta en un ejecutor centralizado (*AlienInvasion*) que orquesta el ciclo de vida del juego mediante un bucle de ejecución de alta precisión basado en señales y ranuras (signals & slots). Este orquestador coordina la gestión de estados, el procesamiento de entradas asíncronas y la actualización determinista de las coordenadas espaciales para todas las entidades activas.

La implementación permite que cada componente —desde la entidad individual del jugador (*Ship*) hasta la lógica de enjambre de la flota enemiga (*Alien*)— opere como una unidad autónoma encapsulada. Esta autonomía, bajo un entorno controlado, permite que la lógica matemática y cinemática defina el comportamiento del sistema, emulando interacciones físicas complejas mediante la resolución de colisiones y la gestión dinámica de recursos en memoria.

METODOLOGIA DE DESARROLLO Y GESTION DE VERSIONES

Para garantizar la integridad del proyecto y permitir un despliegue iterativo y seguro, el desarrollo se llevó a cabo siguiendo una metodología de trabajo basada en el Control de Versiones (Git). Este enfoque permitió una evolución estructurada del código, facilitando la implementación de cambios modulares sin comprometer la estabilidad del sistema:

- **Desarrollo Iterativo:** El proyecto se desglosó en hitos técnicos específicos, permitiendo una validación constante de la lógica física antes de avanzar a la capa visual o de control.
- **Gestión de Cambios y Trazabilidad:** El uso de Git como repositorio remoto en GitHub permitió llevar un historial detallado de las actualizaciones, garantizando que cada ajuste en la calibración de parámetros o definición de clases fuera rastreable y reversible.
- **Sincronización Multi-entorno:** La metodología facilitó la integración continua entre el entorno de desarrollo local (con Qt6) y el repositorio remoto, asegurando la coherencia del estado del código en todo momento.
- **Control de Calidad:** La estructura de "Borrón y cuenta nueva" aplicada en etapas críticas garantizó un repositorio limpio, eliminando residuos de compilación y conflictos de merge que pudieran afectar el rendimiento del ejecutor.

ARQUITECTURA DEL SISTEMA: ESTRATEGIA DE SEPARACION DE RESPONSABILIDADES

El diseño del motor *Alien Invasion* se fundamenta en una estrategia de Desacoplamiento de Componentes, transformando un problema complejo en un ecosistema de módulos especializados. En lugar de una implementación monolítica, el proyecto adopta una Arquitectura Orientada a Objetos que maximiza la mantenibilidad y la escalabilidad del sistema.

A continuación, se detalla la matriz de responsabilidades que define la interacción entre los distintos módulos del motor:

Clase	Modulo	Responsabilidad
AlienInvasion	alien_invasion.cpp	Orquestador Central: Gestiona el ciclo de vida, la inicialización de recursos y el bucle principal de ejecución (Game Loop).
Settings	settings.cpp	Repositorio de Configuración: Almacena constantes físicas y parámetros globales para garantizar la coherencia en la simulación.
Ship	ship.cpp	Entidad Jugador: Encapsula la lógica de movimiento, la gestión de la posición y el renderizado de la nave.
Alien	alien.cpp	Entidad Enemiga: Define el comportamiento autónomo de los invasores y su lógica de traslación espacial.
Bullet	bullet.cpp	Cinemática de Proyectiles: Gestiona la trayectoria, el estado de disparo y la limpieza de memoria tras el impacto o salida de escena.
GameStats	game_stats.cpp	Máquina de Estados: Centraliza el seguimiento de variables críticas como puntuación, nivel, vidas y estado activo/inactivo.
PlayButton	button.cpp	Interfaz de Control: Gestiona los eventos de interacción del usuario para la transición de estados del juego.
Scoreboard	scoreboard.cpp	Capa de Visualización: Responsable de la representación gráfica síncrona de las estadísticas del sistema ante el usuario.

La Clase AlienInvasion (Ejecutor)

La clase **AlienInvasion** actúa como el núcleo ejecutor del sistema, encapsulando la lógica de control del ciclo de vida y la gestión de recursos dinámicos. Su implementación se adhiere a las mejores prácticas de programación defensiva y gestión de memoria en C++:

- **Gestión de Memoria Determinista:** El ciclo de vida de los objetos (**Settings**, **GameStats**, y los contenedores de entidades **Alien** y **Bullet**) es gestionado explícitamente. El constructor realiza la asignación de recursos en memoria mediante el operador `new`, mientras que el destructor implementa una rutina de limpieza profunda (**Deep Cleanup**). Este proceso asegura que cada puntero alojado en los vectores sea liberado correctamente, garantizando la ausencia de fugas de memoria (**memory leaks**) —un requisito crítico para la estabilidad en sistemas de tiempo real.

- **Gestión del Bucle Principal (Main Loop):** El control de flujo se basa en un mecanismo de estado booleano (`bool game_active`). Este interruptor actúa como la condición de evaluación central, permitiendo que el motor determine si debe procesar la lógica de actualización, colisiones y renderizado o mantenerse en un estado de espera, garantizando un control granular sobre el tiempo de ejecución de la simulación.

La Clase `Settings`

La clase `Settings` constituye el pilar de configuración global del motor, centralizando todos los parámetros físicos y constantes del entorno de simulación. Su diseño está orientado a la optimización del rendimiento y la integridad de los datos durante la ejecución.

- **Arquitectura de Inmutabilidad:** Diseñada para actuar como un contenedor de configuración centralizado, esta clase garantiza la consistencia del estado global. Su implementación permite ser compartida entre las distintas entidades del juego mediante paso por referencia constante (`const Settings&`), eliminando la necesidad de realizar copias de memoria redundantes, lo que optimiza significativamente el uso del stack y la eficiencia del ejecutor.
- **Tipado Estricto y Determinismo:** Todos los parámetros —como velocidades, dimensiones de los bounding boxes y escalas de dificultad— están definidos con un tipado explícito (e.g., `float`, `int`, `QColor`). El uso de sufijos de precisión (como `f` para valores de punto flotante) asegura una gestión determinista de la cinemática, permitiendo que la calibración de parámetros —como la `bulletSpeed`— sea predecible y ajustable para alcanzar un comportamiento óptimo del motor durante la ejecución preliminar y las fases de escalabilidad de dificultad.
- **Escalabilidad Dinámica:** Además de servir como un repositorio estático, `Settings` encapsula métodos de escalado dinámico que permiten modificar las variables físicas en tiempo de ejecución (como el incremento de velocidad al cambiar de nivel), permitiendo que el motor ajuste su dificultad sin necesidad de reinstanciar objetos.

Diseño de Entidades: `Ship`, `Alien` y `Bullet`

La arquitectura del motor opta por un diseño de entidades especializadas con alta cohesión, donde cada clase encapsula su propia lógica cinemática y de estado, permitiendo una gestión eficiente y directa desde el orquestador principal (`AlienInvasion`).

- **Especialización de Componentes:** En lugar de una capa de abstracción polimórfica, el sistema utiliza clases concretas para `Ship`, `Alien` y `Bullet`. Esto permite que cada entidad posea atributos y métodos optimizados exclusivamente para su rol (e.g., la lógica de detección de bordes propia de `Alien` o el manejo de estados de movimiento de `Ship`).
- **Gestión Directa por el Ejecutor:** La clase `AlienInvasion` mantiene contenedores específicos (`std::vector<Bullet*>` y `std::vector<Alien*>`) para cada tipo de entidad. Esta aproximación permite una gestión de memoria determinista y un acceso directo a los métodos de cada clase, evitando el sobrecosto (overhead) que implica la búsqueda de tablas virtuales (vtable) en tiempo de ejecución.
- **Control de Estado Robusto:** Cada entidad es responsable de su propio ciclo de actualización (`actualizar_posicion()`). El ejecutor, al conocer el tipo de dato de cada contenedor, invoca los

métodos de forma directa, lo que garantiza que la lógica de colisiones y el movimiento sean evaluados con la mayor eficiencia posible dentro del bucle principal.

- **Integridad de Tipos:** Este diseño elimina la complejidad de jerarquías profundas y permite que el compilador realice optimizaciones específicas para cada tipo de objeto, manteniendo la estructura del código clara y fácil de depurar ante posibles errores de lógica en las fronteras de los objetos.

ESTRUCUTURA DE DEPENDENCIAS Y ACOMPLAMIENTO

La arquitectura del software se rige por un modelo de Alta Cohesión y Bajo Acoplamiento, donde se elimina la herencia cruzada redundante y se prioriza la composición de objetos y la inyección de dependencias por punteros. El sistema se subdivide en tres niveles de jerarquía operacional:

El Núcleo Orquestador (Clase Dominante): AlienInvasion

No es simplemente una ventana gráfica; actúa como el Controlador Central del patrón arquitectónico. Es la única entidad con acoplamiento global (conoce la existencia de todos los componentes) y su responsabilidad matemática es la gestión del Ciclo de Vida (creación, actualización cinemática a $\Delta t \approx 16$ ms y destrucción en la memoria RAM).

Mantiene bajo su dominio exclusivo:

- **Entidades Singulares (Composición Fuerte):** * Ship: Instancia única que representa el centroide del jugador.
 - **GameStats:** La máquina de estados finitos que modela las variables de control (puntuación, nivel y vectores de vida).
 - **Scoreboard:** El despachador síncrono visual encargado de transformar los datos numéricos de GameStats en vectores tipográficos.
 - **Scoreboard:** Objeto visual que lee GameStats.
 - **PlayButton:** Elemento de interfaz interactivo que interrumpe la máquina de estados.
- **Colecciones Dinámicas:**
 - **std::vector<Alien*>** y **std::vector<Bullet*>**: Vectores indexados que reservan memoria contigua en la RAM (reserve) para garantizar un recorrido $O(N)$ de alta velocidad en la memoria caché de la CPU durante el bucle físico.

La Frontera de Configuración (Dependencia Universal): Settings

Es una clase estática de datos monolíticos que encapsula las condiciones de frontera espaciales y las constantes cinemáticas del universo del juego.

- **Mecanismo de Inyección:** En lugar de apelar a variables globales (una mala práctica de ingeniería) o a herencia múltiple (que duplicaría el peso en bytes de los objetos), Settings se propaga mediante Inyección de Dependencias por Punteros a través de los constructores de Ship, Bullet, Alien, GameStats y Scoreboard.
- **Impacto en Memoria:** Esto garantiza que todas las entidades del juego apunten a una única dirección de memoria común de apenas 8 bytes, asegurando que cualquier cambio paramétrico (como el



escalamiento de velocidad al cambiar de nivel) se refleje instantáneamente en todo el sistema sin redundancia de datos.

Las Clases Satélites (Entidades Dependientes)

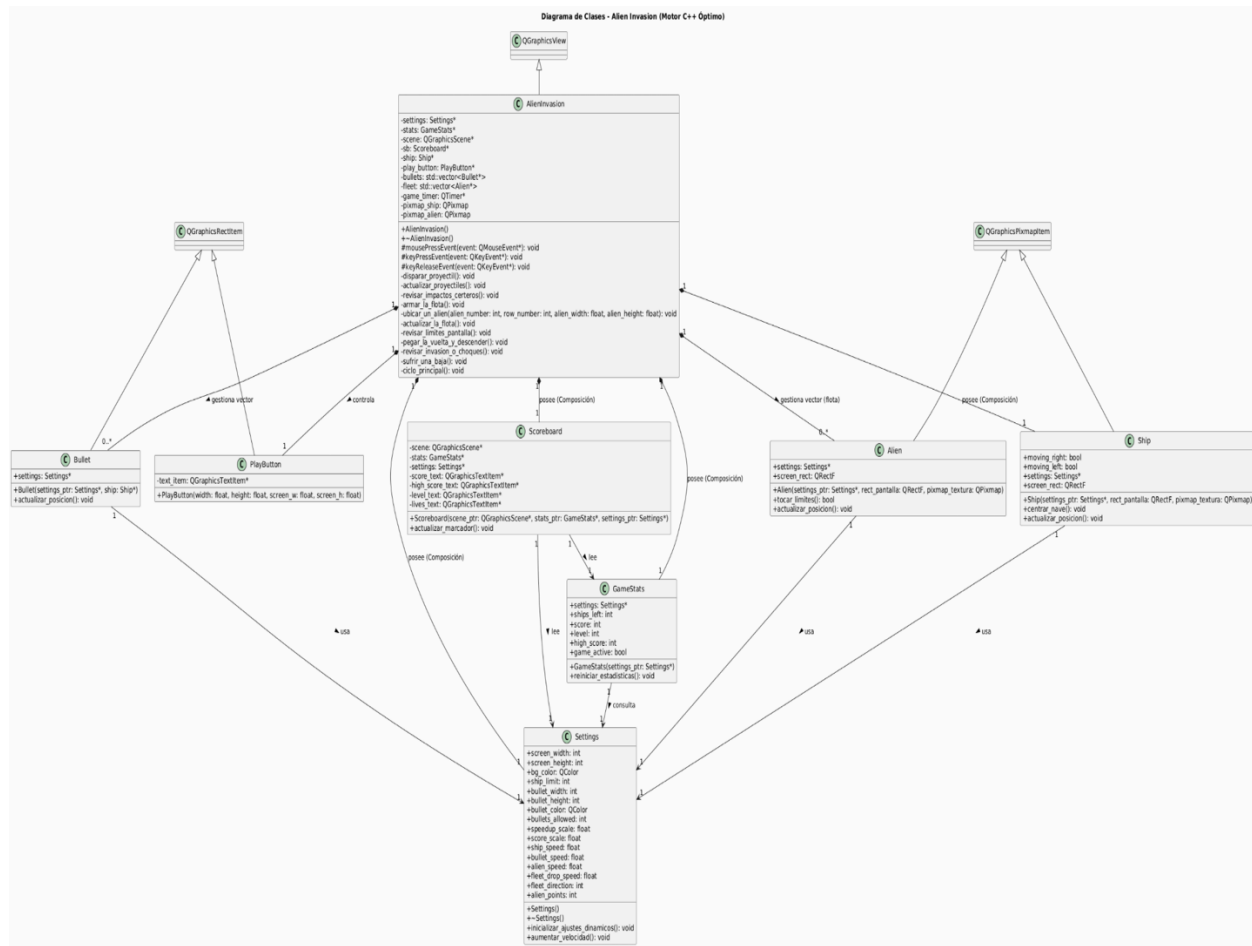
Clases como Ship, Alien y Bullet ocupan el escalón más bajo de la jerarquía. Son clases ciegas entre sí: una bala no sabe que existe un alien, y un alien no sabe que existe una nave. Solo conocen sus propias geometrías y las reglas físicas que extraen de Settings. Esto permite que si necesitas modificar la física de un proyectil, lo hagas de forma aislada sin romper el comportamiento de las demás entidades.

El Grafo de Dependencias

Para modelar con precisión matemática el flujo de acoplamiento de la aplicación, la estructura se proyecta como un Grafo Dirigido Acíclico (DAG), donde los nodos representan las clases y las aristas (edges) definen la dirección de la inyección de datos y la subordinación de memoria:

- AlienInvasion → [Nodo Raíz / Orquestador Central]: Gobierna mediante aristas de composición y agregación el ciclo de vida de Ship, el `std::vector<Alien*>`, el `std::vector<Bullet*>`, Scoreboard y PlayButton. Es el vértice dominante del sistema.
- Settings → [Sumidero de Consulta / Contexto Global]: Recibe aristas de asociación direccional desde todas las entidades dependientes (Ship, Alien, Bullet, GameStats, Scoreboard). El motor principal se encarga de inyectar la dirección de memoria de este nodo a los demás vértices durante la fase de inicialización.
- Scoreboard → [Nodo de Dependencia Funcional]: Mantiene una relación de dependencia directa hacia GameStats. Su flujo de control está estrictamente acoplado a este nodo para realizar la lectura síncrona de las variables dinámicas del estado en cada ciclo de actualización.

DIAGRAMA DE CLASES



El diagrama de clases de la aplicación Alien Invasion revela una arquitectura orientada a objetos de alta cohesión y bajo acoplamiento. A través de la simbología estándar de modelado UML, se puede evidenciar de manera cuantitativa y espacial cómo interactúan los componentes lógicos del motor de juego y la frontera exacta de su acoplamiento con el framework Qt6.

A continuación, se desglosan los detalles estructurales y las dinámicas de memoria que gobiernan este diseño:

El Vértice Central de Control y Ciclo de Vida: AlienInvasion

La clase `AlienInvasion` se posiciona en la cúspide del grafo relacional, actuando como el nodo orquestador dominante.

- **Mecanismo de Composición Fuerte:** Los rombos negros orientados hacia `Settings`, `GameStats`, `Scoreboard` y `Ship` indican una relación de composición directa (1:1). Esto significa que `AlienInvasion` es el dueño absoluto del ciclo de vida de estas instancias en la memoria RAM: las crea al inicializarse el sistema y garantiza su liberación de memoria física mediante el operador `delete` en su destructor.

- **Agregación Matricial Dinámica:** Las conexiones hacia las clases `Bullet` y `Alien` presentan una cardinalidad de cero a muchos (0..*). Esto modela el almacenamiento indexado de los proyectiles y la flota invasora dentro de vectores estándar de C++ (`std::vector<Item*>`). El diagrama expone cómo el motor gestiona de manera eficiente colecciones de tamaño variable en tiempo de ejecución para el procesamiento de colisiones y actualizaciones físicas.

El Patrón de Contexto Unificado: `Settings`

Uno de los puntos estéticamente más robustos del diagrama es el efecto embudo que converge en la clase `Settings` en la base del gráfico.

- **Inyección de Dependencias en lugar de Herencia:** En lugar de apelar a jerarquías verticales o herencia múltiple (lo que duplicaría de forma masiva el uso de bytes por cada objeto en la RAM), las clases satélites (`Bullet`, `Alien`, `Ship`, `GameStats` y `Scoreboard`) se conectan a `Settings` mediante flechas de Asociación Simple.
- **Compartición Síncrona de Datos:** Cada una de estas entidades almacena internamente un puntero de solo 8 bytes que apunta a la misma y única dirección de memoria donde reside `Settings`. Esto asegura que todas las constantes físicas del universo del juego (velocidades, anchos de pantalla, límites) se centralicen en un tablero de control único, permitiendo que modificaciones dinámicas (como el escalamiento de la velocidad en transiciones de nivel) afecten a todo el sistema en tiempo real y con costo computacional

Desacoplamiento de la Interfaz y el Estado: `Scoreboard` y `GameStats`

La relación triangular en la zona media inferior del diagrama ilustra la separación de responsabilidades entre el estado interno y la representación visual:

- La clase `GameStats` actúa de manera pura como una máquina de estados finitos que almacena las variables numéricas y lógicas críticas (score, level, ships_left).
- La clase `Scoreboard` mantiene una relación de dependencia y lectura hacia `GameStats`. Al estar separadas, el diseño cumple con el principio de diseño de software de aislamiento: la interfaz visual solo consulta los datos para redibujar los textos tipográficos, pero no interfiere con las reglas lógicas que alteran las puntuaciones o la progresión del juego.

La Frontera Externa del Framework (`QGraphicsView` e `Items`)

El diagrama delimita con total claridad los componentes locales de la lógica de negocio frente a las clases heredadas del framework gráfico de Qt6 (representadas por las cajas en los extremos superiores):

- `AlienInvasion` hereda verticalmente de `QGraphicsView` para transformarse en el lienzo de visualización que interactúa con el hardware y los eventos de teclado/mouse.
- Las entidades móviles y de interfaz se bifurcan según su naturaleza matemática vectorial o rasterizada: `Ship` y `Alien` heredan de `QGraphicsPixmapItem` para heredar las transformaciones espaciales basadas en mapas de bits, mientras que `Bullet` y `PlayButton` heredan de `QGraphicsRectItem` para operar como polígonos puros de renderizado vectorial rápido.

BIBLIOGRAFIA

- Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley. (2010).
- Gregory, J. (2018). *Game Engine Architecture* (3rd ed.). CRC Press.
- Millington, I. (2010). *Game Physics Engine Development: How to Build a Robust Commercial-Grade Physics Engine for your Game* (2nd ed.). Morgan Kaufmann.
- Deitel, P. J., & Deitel, H. M. (2017). *C++ How to Program* (10th ed.). Pearson.
- Prata, S. (2011). *C++ Primer Plus* (6th ed.). Addison-Wesley.

UdeA